Software Architecture

Hand-in Assignment

Week 8

Instructions:

- Submit your solutions through GitHub Classroom and **remember** to submit a link to your repository in BrightSpace. Otherwise, I will have a hard time distinguishing between the different groups.
- When submitting, you should push your solutions to the existing repository. In this assignment, you will primarily need to answer questions in a Markdown file and modify the existing `chat_client.cpp`.
- **Note on system setup:** For this assignment, you should use the exact same setup as you did in Assignment 6 when we installed Boost. Windows users must use their Ubuntu terminal (WSL), and Mac users should use Homebrew. You do not need to use a Virtual Machine (VM).
- You **must** add comments in your programs to explain your code. Any solutions submitted without any comments will not be graded.

Description

During this exercise you will implement a simple peer-to-peer chat program that allows you to send simple text messages to other clients on your network. The exercise is split into two parts:

1. Warmup questions
2. Modifying existing program

Note that both of these are mandatory parts of the assignment.

Installing ZMQ

Depending on your operating system, there are different ways to install the ZeroMQ (ZMQ) library and its C++ bindings (cppzmq). Please follow the instructions that match your system:

Mac (Homebrew)

If you are on a Mac, you can easily install the packages using Homebrew. Open your terminal and run the following commands:

```
brew install zeromq
brew install cppzmq
```

Windows (WSL / Ubuntu) & Linux

If you are on Windows, open your Ubuntu terminal (WSL). If you are on a native Linux machine, open your standard terminal. We will install the library from source. Run the following commands:

```
# install libzmq from source
git clone https://github.com/zeromq/libzmq.git
cd libzmq
mkdir build
```

```
cd build
cmake ..
cmake --build .
sudo cmake --install .
cd ../..

# install cppzmq, (C++ bindings to libzmq)
git clone https://github.com/zeromq/cppzmq
cd cppzmq
mkdir build
cd build
cmake ..
cmake --build .
sudo cmake --install .
cd ../..
```

After running the appropriate commands for your system, both `zmq` and `cppzmq` should be installed and ready to use.

Exercise

Discussion: Network Architectures

Create a file `discussion.md` in the repo and answer the following questions:

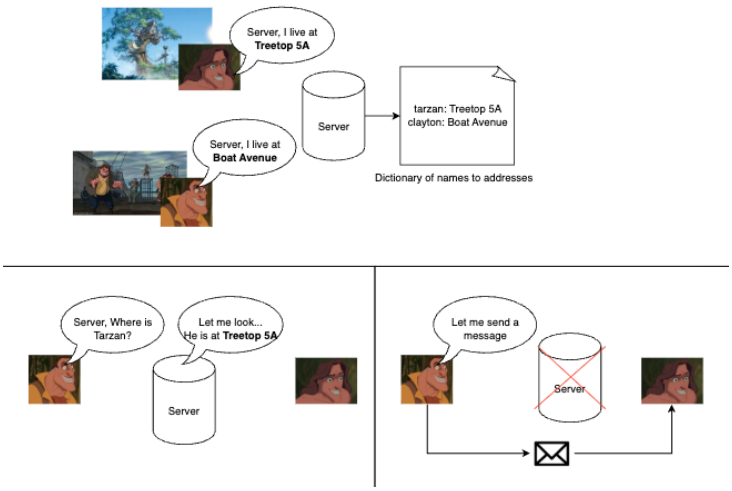
1. **What is an IP address? (100 words)**
   - Why do we need IP addresses?
   - What is the relation between TCP and IP?
   - What is a socket? Have you heard of different types of sockets?
   - What is up with the address: 127.0.0.1?

2. **What does a client-server architecture mean? (150 words)**
   - Give a concrete example of where you would encounter this in real life.
   - How do you decide who is the server and who is the client?

3. **What is the difference between client-server architecture and the broker architecture? (100 words)**
   - What are the benefits to the broker pattern?
   - What could be potential downsides?

4. **What does Peer-to-Peer mean? (150 words)**
   - What are the main characteristics of this architecture?
   - Have you encountered it anywhere online?
   - Does Peer-to-Peer applications mean that there are no servers and only clients?
   - What benefits could there be to using this pattern?
   - What downsides could there be to this pattern?

Programming: Chat Program



1. Inspect the following files `chat_client.cpp` and `chat_server.cpp`. Try to relate the concepts to the image above. Focus on the high level structure like where data structures are initialized and where the reading and writing to the socket occurs.

2. Compile the programs and start 1 server and two clients, as shown below:

```
./chat_server 127.0.0.1
```

Then start the first client in a new terminal:

```
./chat_client 127.0.0.1 tarzan 8000
```

Start the second client in a new terminal:

```
./chat_client 127.0.0.1 clayton 9000
```

The first argument `127.0.0.1` is the ip address of the server. The client program accepts two additional arguments, the first being the name of the client and the second being the port number used by the specific client.

The output in the server terminal should be:

```
binding welcome socket to address: tcp://127.0.0.1:5000
awaiting registration of chat clients
received handshake from: 'clayton', with endpoint: 'tcp
    ://127.0.0.1:8000'
received handshake from: 'tarzan', with endpoint: 'tcp
    ://127.0.0.1:9000'
```

Note: To enable chat between multiple computers you could start the server on a computer that is accessible by all computers on the network and change the argument appropriately ***(you dont have to do this)****.*

3. Try to send a message from Clayton to Tarzan by typing the following into Clayton's terminal:

```
tarzan,Hi meet me in the dark woods <3 - Jane!
```

Clayton's terminal:

```
tarzan,hi meet me in the dark woods <3 - Jane
address of recipient: 'tarzan' is not known to the client, asking
    server
server resolved address of: 'tarzan', address is: 'tcp
    ://127.0.0.1:9000'
sending message: 'hi meet me in the dark woods <3 - Jane' to: '
    tarzan'
```

Tarzan's terminal should show:

```
...
received message: 'hi meet me in the dark woods <3 - Jane'
```

4. Clearly, Clayton is trying to lure Tarzan into a trap. Modify the program `chat_client.cpp` such that the name of the sender is shown when receiving the message. After modifying the program the message received by Tarzan should look like:

```
...
received message from 'clayton': 'hi meet me in the dark woods <3
    - Jane'
```